

생성형 AI와 LLM 기반 AI 에이전트 개발

LangGraph 소개



왜 LangGraph가 필요한가?

LangGraph 소개

- LangChain으로 만든 에이전트들은 기본적으로 선형 구조.
사용자 입력 → LLM 판단 → Tool 실행 → 응답 출력의 순서가 고정되어 있음.
- 실제 복잡한 에이전트에서는 이런 상황이 생김:
 - 조건에 따라 다른 경로로 분기해야 할 때
 - 특정 단계를 반복해야 할 때
 - 여러 에이전트가 협업해야 할 때
 - 작업 중간에 사람의 확인(Human-in-the-loop)이 필요할 때

```
# LangChain Chain 방식 — 순서가 고정된 선형 구조
chain = prompt | llm | parser
# 항상 같은 순서로만 실행됨. 조건 분기나 반복 불가
```

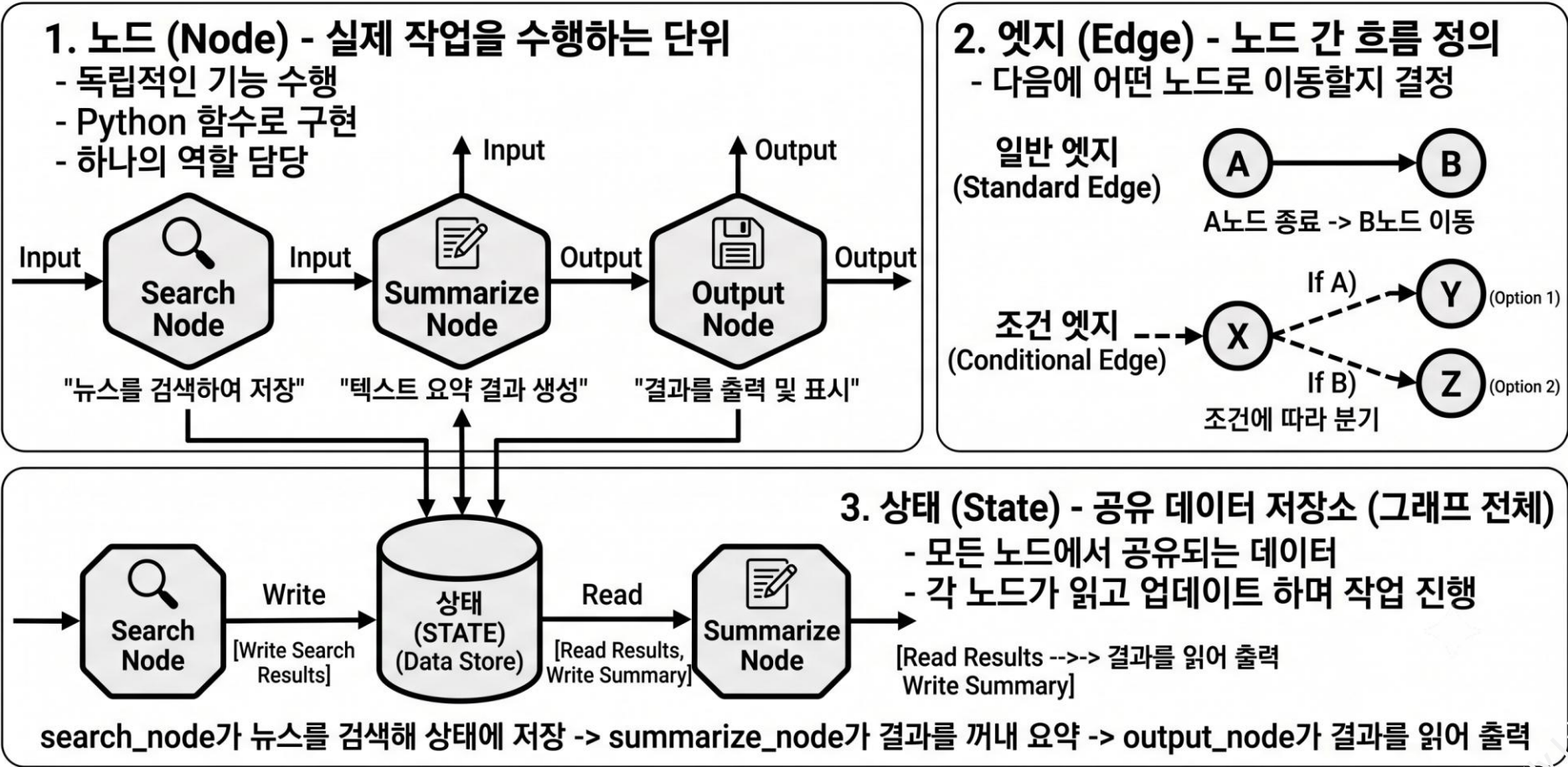
```
# LangGraph 방식 — 조건에 따라 다른 경로로 분기 가능
# 뉴스 검색 필요? → search_node로
# 요약만 필요? → summarize_node로
# 추가 정보 필요? → 다시 search_node로 반복
```

- LangGraph는 이런 복잡한 Workflow를 노드(Node)와 엣지(Edge)로 시각적으로 설계할 수 있게 해주는 프레임워크.

LangGraph의 핵심 개념

LangGraph 소개

- LangGraph는 에이전트의 실행 흐름을 그래프(Graph) 구조로 표현함. 그래프는 노드와 엣지로 구성되며, 이 구조를 통해 복잡한 작업 흐름을 시각적으로 설계하고 제어할 수 있음.



LangGraph의 핵심 개념

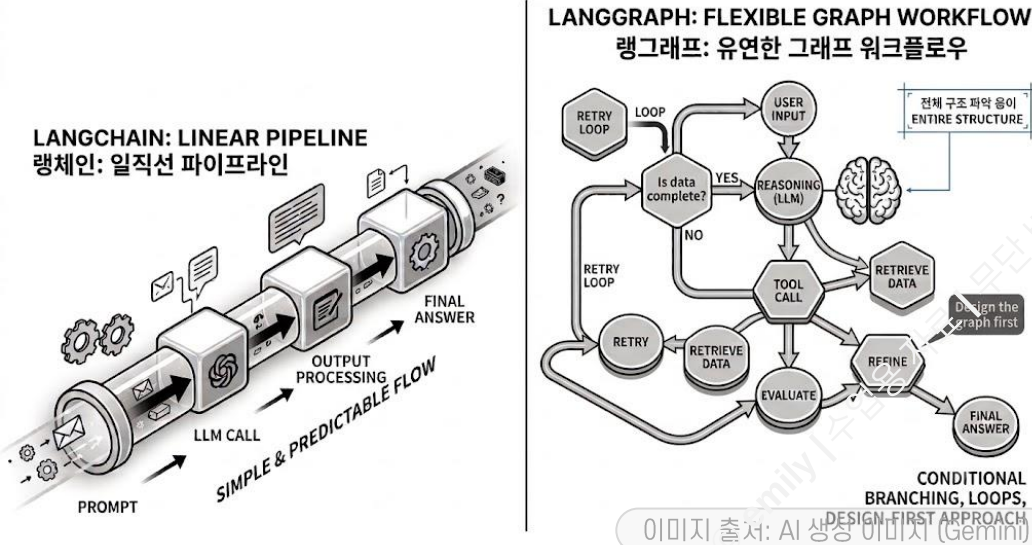
LangGraph 소개

■ 요약

개념	설명	비유
노드 (Node)	실제 작업을 수행하는 단위	공정의 각 작업 단계
엣지 (Edge)	노드 간의 연결 및 실행 순서	작업 단계를 잇는 화살표
조건 엣지 (Conditional Edge)	조건에 따라 다른 노드로 분기	조건에 따른 분기점
상태 (State)	노드 간에 공유되는 데이터	작업 중 계속 업데이트되는 메모장

- LangChain의 Chain이 일직선 파이프라인이라면, LangGraph는 조건 분기와 반복이 가능한 흐름도 (Flowchart)에 가까움.

노드와 엣지로 구성된 그래프를 먼저 설계하고,
그 위에 코드를 얹는 방식으로 개발하기 때문에
복잡한 에이전트일수록 전체 구조를 한눈에 파악하기 쉬움.



상태(State)란?

LangGraph 소개

- 그래프 전체에서 공유되는 데이터 저장소.
각 노드가 실행될 때마다 상태를 읽고 업데이트하면서 작업이 진행됨.

```
from typing import TypedDict, List

# State 정의 — 그래프 전체에서 공유되는 데이터 구조
class AgentState(TypedDict):
    messages: List[str]      # 대화 기록
    keyword: str             # 검색 키워드
    news: str                # 검색된 뉴스
    summary: str             # 요약 결과
    is_done: bool            # 작업 완료 여부
```

- State가 중요한 이유:
 - 각 노드가 독립적으로 실행되면서도 이전 노드의 결과를 참조할 수 있음
 - 작업 중간에 어떤 데이터가 어떻게 변화하는지 추적 가능
 - 조건 분기 시 상태값을 기준으로 다음 노드를 결정

노드와 엣지 설계하기

LangGraph 소개

- 노드는 Python 함수로 구현됨. 각 노드는 현재 상태를 입력받아 작업을 수행하고 업데이트된 상태를 반환.

```
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, END

llm = ChatOpenAI(model="gpt-4o-mini")
# 노드 정의 — 각 노드는 state를 받아 업데이트된 state를 반환
def search_node(state: AgentState) -> AgentState:
    """뉴스를 검색하는 노드"""
    keyword = state["keyword"]
    # 실제 검색 로직
    state["news"] = f"{keyword} 관련 뉴스 검색 결과"
    return state
def summarize_node(state: AgentState) -> AgentState:
    """뉴스를 요약하는 노드"""
    news = state["news"]
    response = llm.invoke(f"다음 뉴스를 요약해줘: {news}")
    state["summary"] = response.content
    return state
def end_node(state: AgentState) -> AgentState:
    """작업을 마무리하는 노드"""
    state["is_done"] = True
    return state
```

조건 엣지(Conditional Edge) 설계

LangGraph 소개

- 조건 엣지(Conditional Edge)는 상태값에 따라 다음에 실행할 노드를 동적으로 결정하는 엣지. 일반 엣지가 항상 같은 노드로 이동하는 것과 달리, 조건 엣지는 조건 분기 함수의 반환값에 따라 서로 다른 노드로 이동할 수 있음.

```
# 조건 분기 함수 — 상태를 보고 다음 노드 이름을 반환
def should_continue(state: AgentState) -> str:
    """뉴스 검색 결과가 있으면 요약으로, 없으면 종료"""
    if state["news"]:
        return "summarize"          # → summarize_node로 이동
    else:
        return "end"                # → end_node로 이동

# 그래프 구성
graph = StateGraph(AgentState)

# 노드 추가
graph.add_node("search", search_node)
graph.add_node("summarize", summarize_node)
graph.add_node("end", end_node)
...
```


조건 엣지(Conditional Edge) 설계

LangGraph 소개

```
...
# 엣지 추가
graph.set_entry_point("search")
# ↑ 그래프의 시작점 설정

graph.add_conditional_edges(
    "search",                                     # 출발 노드
    should_continue,                             # 조건 분기 함수 — 상태를 보고 다음 노드 결정
    {
        "summarize": "summarize",               # "summarize" 반환 시 summarize 노드로
        "end": "end"                             # "end" 반환 시 end 노드로
    }
)

graph.add_edge("summarize", END)
# ↑ summarize 노드 완료 후 그래프 종료 (일반 엣지)

# 그래프 컴파일 및 실행
app = graph.compile()
result = app.invoke({"keyword": "인공지능", "news": "", "summary": "", "is_done": False})
print(result["summary"])
```

LangChain vs LangGraph 비교

LangGraph 소개

- **LangGraph가 항상 더 좋은 것은 아님.**
단순한 에이전트는 LangChain Chain으로 충분하며, 복잡한 Workflow가 필요할 때 LangGraph를 도입하는 것이 효율적.

	LangChain	LangGraph
구조	선형 파이프라인	그래프 (노드/엣지)
흐름 제어	순서 고정	조건 분기·반복 가능
상태 관리	Memory로 대화 기록만 유지	State로 모든 데이터 공유·추적
적합한 경우	단순하고 순서가 고정된 작업	복잡한 조건 분기·반복·멀티에이전트
복잡도	상대적으로 단순	상대적으로 복잡

실습: 간단한 LangGraph 흐름 만들기

LangGraph 소개

- **실습의 목적:**
 - AI가 생성한 코드를 실행해보면서 노드, 엣지, 상태, 조건 분기가 실제로 어떻게 작동하는지 이해
 - LangChain Chain과 LangGraph의 구조적 차이를 직접 체감
- **아래 프롬프트를 평소 사용하는 AI(ChatGPT, Claude 등)에 입력하고, 생성된 코드를 VSCode에서 실행해보세요.**

💬 프롬프트 예시

개발 환경은 Miniconda 가상환경(Python 3.11)이며 VSCode 터미널에서 실행합니다.
Python으로 LangGraph를 활용한 간단한 뉴스 검색·요약 그래프를 만들어줘. 조건은 다음과 같아.

- .env 파일에서 OpenAI API Key를 불러오도록 구현
- TypedDict로 AgentState 정의 (keyword, news, summary, is_done 필드 포함)
- 아래 세 가지 노드 구현:
 - search_node: 키워드로 뉴스 검색 (임시 반환값 사용)
 - summarize_node: 검색된 뉴스를 LLM으로 요약
 - end_node: 작업 완료 표시
- 조건 분기: 뉴스 검색 결과가 있으면 summarize_node로, 없으면 end_node로
- StateGraph로 그래프 구성 후 컴파일
- gpt-4o-mini 모델 사용
- 사용자가 키워드를 입력하면 그래프 실행 후 요약 결과 출력

실습: 간단한 LangGraph 흐름 만들기

LangGraph 소개

■ 실습의 목적:

- AI가 생성한 코드를 실행해보면서 노드, 엣지, 상태, 조건 분기가 실제로 어떻게 작동하는지 이해
- LangChain Chain과 LangGraph의 구조적 차이를 직접 체감

🔍 확인해 볼 것

- 생성된 코드에서 노드, 엣지, 조건 분기가 각각 어느 부분인지 찾아보기
- 상태(State)가 각 노드를 거치면서 어떻게 업데이트되는지 확인
- LangChain Chain 코드와 구조적으로 어떻게 다른지 비교

AI Agent 심화



ReAct 패턴이란?

AI Agent 심화

- **ReAct(Reasoning + Acting):** LLM이 생각(Reasoning)과 행동(Acting)을 번갈아 반복하면서 문제를 해결하는 에이전트 설계 패턴. 2022년 Google Research에서 발표한 논문에서 제안된 개념으로, 현재 대부분의 AI 에이전트가 이 패턴을 기반으로 동작함.
- **ReAct 패턴의 실행 흐름:**
 - Thought(생각): LLM이 현재 상황을 분석하고 다음에 무엇을 해야 할지 판단
 - Action(행동): 판단에 따라 Tool을 선택하고 실행
 - Observation(관찰): Tool 실행 결과를 확인
 - 반복: 목표가 달성될 때까지
Thought → Action → Observation을 반복
 - Final Answer: 목표 달성 시 최종 답변 생성

사용자: "최근 AI 관련 뉴스를 검색하고 요약해줘"

Thought: 뉴스를 검색해야 한다. search_news Tool을 사용하자.

Action: search_news("AI")

Observation: "OpenAI 새 모델 발표, Google Gemini 업데이트..."

Thought: 검색 결과가 나왔다. 이제 요약해야 한다.

Action: summarize(검색결과)

Observation: "AI 분야에서 주요 모델 업데이트가 있었음..."

Final Answer: "최근 AI 뉴스 요약: ..."

ReAct 패턴과 LangGraph의 연결

AI Agent 심화

- **LangGraph의 노드/엣지 구조가 실제로 ReAct 패턴을 구현하는 방식임.**
각 구성요소가 어떻게 대응되는지 확인.

ReAct 패턴	LangGraph 구현
Thought (생각)	LLM 노드 — 상황 판단 및 다음 행동 결정
Action (행동)	Tool 노드 — 선택된 Tool 실행
Observation (관찰)	상태(State) 업데이트 — Tool 실행 결과 저장
반복	조건 엣지 — 목표 달성 여부에 따라 반복 또는 종료
Final Answer	종료 노드 — 최종 답변 생성

- 즉, LangGraph로 에이전트를 설계한다는 것은 ReAct 패턴을 노드와 엣지로 시각적으로 구현하는 것.

LangGraph로 ReAct 에이전트 설계하기

AI Agent 심화

- LangGraph 기본 구조(노드 → 엣지 → 조건 엣지)는 동일하게 사용됨.
ReAct 패턴에서 추가되는 핵심 차이는 act_node 완료 후 다시 think_node로 돌아오는 순환 구조.
기존 그래프는 한 방향으로만 흘렀다면, ReAct 에이전트는 목표가 달성될 때까지 think → act → think → act를 반복함.

	LangGraph	ReAct 에이전트
흐름	단방향 (시작 → 종료)	순환 (think → act → think 반복)
종료 조건	마지막 노드 도달 시	LLM이 목표 달성을 판단할 때
핵심 추가 요소	-	act → think 순환 엣지

- 예시 코드(1)

```
from langgraph.graph import StateGraph, END
from langchain_openai import ChatOpenAI
from typing import TypedDict, List
class AgentState(TypedDict):
    messages: List[str]
    next_action: str
    tool_result: str
    ...
```

```
# 대화 기록
# 다음 실행할 행동 ("tool" 또는 "end")
# Tool 실행 결과
```


LangGraph로 ReAct 에이전트 설계하기

AI Agent 심화

■ 예시 코드(2)

```
...
llm = ChatOpenAI(model="gpt-4o-mini")

def think_node(state: AgentState) -> AgentState:
    """Thought: LLM이 상황을 판단하고 다음 행동을 결정하는 노드"""
    response = llm.invoke(state["messages"])
    # LLM 응답에 Tool 호출이 필요하면 "tool", 완료면 "end"
    state["next_action"] = "tool" if "검색" in response.content else "end"
    state["messages"].append(response.content)
    return state

def act_node(state: AgentState) -> AgentState:
    """Action: Tool을 실행하는 노드"""
    # Tool 실행 후 결과를 상태에 저장
    state["tool_result"] = "Tool 실행 결과"
    state["messages"].append(f"Observation: {state['tool_result']}")
    return state
```

LangGraph로 ReAct 에이전트 설계하기

AI Agent 심화

■ 예시 코드(3)

```
...
def should_continue(state: AgentState) -> str:
    """Observation: Tool 결과를 보고 반복할지 종료할지 결정"""
    return state["next_action"]
    # "tool" → act_node로 돌아가 반복
    # "end" → 그래프 종료

# 그래프 구성
graph = StateGraph(AgentState)
graph.add_node("think", think_node)
graph.add_node("act", act_node)
graph.set_entry_point("think")
graph.add_conditional_edges("think", should_continue, {
    "tool": "act",          # Tool 실행 필요 → act_node로
    "end": END              # 완료 → 그래프 종료
})
graph.add_edge("act", "think")
# ↑ Tool 실행 후 다시 think_node로 돌아와 반복
# act_node 완료 후 다시 think_node로 돌아오는 순환 구조
# 이 한 줄이 ReAct 패턴의 "반복" 을 구현하는 부분

app = graph.compile()
```

수업용 자료 | 무단 배포 금지

LangSmith란?

AI Agent 심화

- **LangChain/LangGraph로 만든 에이전트를 모니터링하고 디버깅할 수 있는 도구.**
에이전트가 어떤 판단을 내리고, 어떤 Tool을 호출했으며, 각 단계에서 얼마나 많은 토큰을 사용했는지 시각적으로 확인 가능.
- **LangSmith가 필요한 이유:**
 - 에이전트가 복잡해질수록 어느 단계에서 문제가 생겼는지 파악하기 어려움
 - verbose=True로 터미널 출력을 보는 것보다 훨씬 직관적으로 흐름을 파악 가능
 - 토큰 사용량과 비용을 추적해 효율적인 에이전트 설계 가능
 - 실제 운영 환경에서 에이전트 성능을 지속적으로 모니터링 가능

LangSmith 설정하기

AI Agent 심화

- LangSmith는 무료로 사용할 수 있으며, 환경 변수 설정만으로 기존 LangChain/LangGraph 코드에 바로 연동 가능.
- 설정 순서:
 1. smith.langchain.com 접속 및 회원가입
 2. 대시보드에 Settings에서 API Key 발급
 3. .env 파일에 아래 내용 추가

.env 파일에 추가

LANGCHAIN_TRACING_V2=true

LANGCHAIN_API_KEY=ls_XXXXXXXXXXXXXXXXXXXXXXXXXXXX

LANGCHAIN_PROJECT=my-agent-project # 프로젝트 이름 (자유롭게 설정)

기존 코드에 추가 설정 없이 .env만 설정하면 자동으로 추적 시작

from dotenv import load_dotenv

load_dotenv()

이후 LangChain/LangGraph 코드 실행 시

LangSmith 대시보드에서 실행 기록이 자동으로 쌓임

LangSmith로 확인할 수 있는 것들

AI Agent 심화

- LangSmith 대시보드에서 확인 가능한 주요 정보:

항목	설명
실행 흐름	각 노드가 어떤 순서로 실행됐는지 시각적으로 확인
입출력 데이터	각 노드의 입력값과 출력값 상세 확인
토큰 사용량	각 LLM 호출에서 사용된 토큰 수와 비용
실행 시간	각 노드의 실행 시간 및 전체 응답 시간
오류 추적	어느 노드에서 오류가 발생했는지 즉시 파악

- 실제 에이전트를 개발할 때 LangSmith는 단순한 모니터링 도구를 넘어, 에이전트의 성능을 개선하고 비용을 최적화하는 핵심 도구로 활용됨.

실습: ReAct 에이전트 만들고 LangSmith로 모니터링하기

AI Agent 심화

■ 실습의 목적:

- ReAct 패턴(Thought → Action → Observation)이 실제 코드에서 어떻게 구현되는지 이해
- LangSmith 대시보드에서 에이전트의 실행 흐름을 시각적으로 확인
- LangGraph 노드/엣지 구조와 ReAct 패턴이 어떻게 연결되는지 체감
- **아래 프롬프트를 평소 사용하는 AI(ChatGPT, Claude 등)에 입력하고, 생성된 코드를 VSCode에서 실행해보세요.**

💬 프롬프트 예시

개발 환경은 Miniconda 가상환경(Python 3.11)이며 VSCode 터미널에서 실행합니다.
Python으로 LangGraph를 활용한 ReAct 패턴 에이전트를 만들어줘. 조건은 다음과 같아.

- .env 파일에서 OpenAI API Key와 LangSmith API Key를 불러오도록 구현
- TypedDict로 AgentState 정의 (messages, next_action, tool_result 필드 포함)
- think_node: LLM이 상황을 판단하고 다음 행동을 결정
- act_node: 뉴스 검색 Tool 실행 (임시 반환값 사용)
- 조건 엣지: next_action이 "tool"이면 act_node로, "end"이면 그래프 종료
- act_node 완료 후 다시 think_node로 돌아오는 반복 구조
- gpt-4o-mini 모델 사용
- 사용자가 키워드를 입력하면 그래프 실행 후 최종 답변 출력

실습: 간단한 LangGraph 흐름 만들기

AI Agent 심화

■ 실습의 목적:

- ReAct 패턴(Thought → Action → Observation)이 실제 코드에서 어떻게 구현되는지 이해
- LangSmith 대시보드에서 에이전트의 실행 흐름을 시각적으로 확인
- LangGraph 노드/엣지 구조와 ReAct 패턴이 어떻게 연결되는지 체감

🔍 확인해 볼 것

- LangSmith 대시보드에서 Thought → Action → Observation 흐름이 어떻게 시각화되는지 확인
- 각 노드의 입출력 데이터와 토큰 사용량 확인
- 에이전트가 몇 번의 반복 후 최종 답변을 생성하는지 확인

채용 공고 RAG 에이전트



- **RAG(Retrieval-Augmented Generation)**

LLM이 답변을 생성할 때 외부 문서나 데이터베이스에서 관련 정보를 검색(Retrieval)해서 활용하는 방식.
LLM이 학습 데이터에 없는 최신 정보나 특정 문서의 내용을 바탕으로 정확한 답변을 생성할 수 있게 해줌.

- **RAG가 필요한 이유:**

- LLM은 학습 데이터 기준으로만 답변 → 최신 정보나 특정 문서 내용을 모름
- 특정 회사의 채용공고, 내부 문서 등 LLM이 학습하지 않은 데이터 활용 불가
- RAG를 사용하면 외부 문서를 실시간으로 검색해서 LLM에 전달 가능

RAG 없는 경우

사용자: "A회사 채용공고에서 요구하는 기술 스택이 뭐야?"

LLM: "A회사 채용공고에 대한 정보가 없습니다."

RAG 있는 경우

사용자: "A회사 채용공고에서 요구하는 기술 스택이 뭐야?"

→ 채용공고 문서에서 관련 내용 검색

→ 검색 결과를 LLM에 전달

LLM: "A회사 채용공고에서는 Python, SQL, LangChain 경험을 요구합니다."

RAG의 구조

채용 공고 RAG 에이전트

- **RAG는 크게 두 단계로 구성됨.**
 - **1단계: 문서 준비 (인덱싱)**
 - 문서를 작은 단위(chunk)로 분할
 - 각 chunk를 벡터(숫자 배열)로 변환 — 임베딩(Embedding)
 - 벡터 데이터베이스에 저장
 - **2단계: 검색 및 생성 (Runtime)**
 - 사용자 질문을 벡터로 변환
 - 벡터 데이터베이스에서 유사한 chunk 검색
 - 검색된 chunk + 사용자 질문을 LLM에 전달
 - LLM이 검색 결과를 바탕으로 답변 생성

구성요소	역할	실습 활용
문서 로더	채용공고 텍스트 불러오기	TextLoader
텍스트 분할기	문서를 chunk로 분할	RecursiveCharacterTextSplitter
임베딩 모델	텍스트를 벡터로 변환	OpenAIEmbeddings
벡터 데이터베이스	벡터 저장 및 검색	FAISS

에이전트 설계

채용 공고 RAG 에이전트

- **에이전트 이름:** 채용 공고 RAG 에이전트
에이전트 목표: 채용공고 문서를 기반으로 사용자 질문에 정확하게 답변

구성요소	내용
LLM	gpt-4o-mini / 채용공고 분석 전문가 역할
Tool	벡터 데이터베이스 검색 함수
Memory	대화 기록 및 검색 결과 유지
Workflow	질문 입력 → 벡터 검색 → 검색 결과 + 질문 → LLM 답변 생성

RAG 파이프라인 구현

채용 공고 RAG 에이전트

- **RAG 인덱싱 과정의 참조 코드. 실습 프롬프트에 조건으로 포함되어 있으니 구조를 이해하는 데 활용하세요.**

```
from langchain.document_loaders import TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain.vectorstores import FAISS

# 1. 채용공고 문서 로드
# → job_posting.txt 파일에 채용공고 내용을 미리 저장해두고 불러옴
# → encoding="utf-8": 한국어 텍스트가 포함된 경우 반드시 지정
loader = TextLoader("job_posting.txt", encoding="utf-8")
documents = loader.load()      # 파일 전체 내용이 하나의 Document 객체로 로드됨

# 2. 문서를 chunk로 분할
# → LLM은 한 번에 처리할 수 있는 텍스트 길이(컨텍스트 윈도우)가 제한됨
# → 문서 전체를 한 번에 넣는 대신 작은 단위로 잘라서 관리
# → chunk_size=500: 한 chunk의 최대 글자 수
# → chunk_overlap=50: 앞뒤 chunk와 50글자씩 겹치게 해서 문맥이 끊기지 않도록 함
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)
...
```

수업용 자료 | 무단 배포 금지

RAG 파이프라인 구현

채용 공고 RAG 에이전트

- **RAG 인덱싱 과정의 참조 코드. 실습 프롬프트에 조건으로 포함되어 있으니 구조를 이해하는 데 활용하세요.**

```
...
chunks = splitter.split_documents(documents)
# → chunks: 문서가 여러 개의 작은 조각으로 분할된 리스트

# 3. 임베딩 모델로 벡터 변환 후 FAISS에 저장
# → 임베딩: 텍스트를 숫자 배열(벡터)로 변환하는 과정
# → 의미가 유사한 텍스트는 벡터 공간에서 가까운 위치에 놓임
# → FAISS: Facebook에서 개발한 벡터 검색 라이브러리. 로컬에서 무료로 사용 가능
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(chunks, embeddings)
# → vectorstore: 모든 chunk의 벡터가 저장된 데이터베이스

# 4. 검색기 생성
# → k=3: 사용자 질문과 가장 유사한 chunk를 3개 검색
# → k값이 클수록 더 많은 문맥을 LLM에 전달하지만 토큰 소모도 증가
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
```

LangGraph로 RAG 에이전트 구성

채용 공고 RAG 에이전트

- **LangGraph로 RAG 파이프라인을 구성하는 참조 코드. retrieve → generate 흐름에 집중해서 살펴보세요.**

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, List

class RAGState(TypedDict):
    question: str          # 사용자 질문 — 그래프 시작 시 입력
    retrieved_docs: str     # 검색된 문서 내용 — retrieve_node가 채워줌
    answer: str             # 최종 답변 — generate_node가 채워줌

def retrieve_node(state: RAGState) -> RAGState:
    """벡터 데이터베이스에서 관련 문서를 검색하는 노드"""
    # 사용자 질문을 벡터로 변환해서 유사한 chunk 검색
    docs = retriever.invoke(state["question"])

    # 검색된 chunk들을 줄바꿈으로 연결해 하나의 문자열로 합침
    # → LLM에 전달하기 쉬운 형태로 변환
    state["retrieved_docs"] = "\n".join([doc.page_content for doc in docs])
    return state
...
```

LangGraph로 RAG 에이전트 구성

채용 공고 RAG 에이전트

- LangGraph로 RAG 파이프라인을 구성하는 참조 코드. retrieve → generate 흐름에 집중해서 살펴보세요.

```
...
def generate_node(state: RAGState) -> RAGState:
    """검색된 문서를 바탕으로 LLM이 답변을 생성하는 노드"""
    # 검색된 문서 내용과 사용자 질문을 함께 프롬프트에 포함
    # → LLM이 채용공고 내용을 참고해서 정확한 답변을 생성할 수 있음
    prompt = f"""
    다음 채용공고 내용을 참고해서 질문에 답변해줘.

    채용공고 내용:
    {state["retrieved_docs"]}
    # ↑ retrieve_node가 검색한 관련 chunk들이 여기에 삽입됨

    질문: {state["question"]}
    """
    response = llm.invoke(prompt)
    state["answer"] = response.content
    return state
...
```

LangGraph로 RAG 에이전트 구성

채용 공고 RAG 에이전트

- LangGraph로 RAG 파이프라인을 구성하는 참조 코드. retrieve → generate 흐름에 집중해서 살펴보세요.

```
...  
# 그래프 구성  
graph = StateGraph(RAGState)  
graph.add_node("retrieve", retrieve_node)  
graph.add_node("generate", generate_node)  
graph.set_entry_point("retrieve")  
# ↑ 항상 retrieve_node에서 시작 — 먼저 문서를 검색한 뒤 답변 생성  
  
graph.add_edge("retrieve", "generate")  
# ↑ 검색 완료 후 항상 답변 생성 노드로 이동 (조건 없는 일반 엣지)  
# → RAG는 항상 검색 후 생성 순서를 따르므로 조건 엣지가 필요 없음  
  
graph.add_edge("generate", END)  
# ↑ 답변 생성 완료 후 그래프 종료  
  
app = graph.compile()
```


실습: RAG 에이전트 만들기

채용 공고 RAG 에이전트

■ 실습의 목적:

- RAG 구조(문서 준비 → 검색 → 답변 생성)가 실제 코드에서 어떻게 구현되는지 이해
- LangGraph 노드로 RAG 파이프라인을 구성하는 방식 체감
- 자소서 에이전트와 연결해서 실제 채용공고를 활용해보기

🔍 확인해 볼 것

- 생성된 코드에서 인덱싱 단계와 검색·생성 단계가 어느 부분인지 찾아보기
- 채용공고 내용을 바꿔가며 질문에 대한 답변이 어떻게 달라지는지 확인
- LangSmith 대시보드에서 retrieve → generate 흐름 확인

실습: RAG 에이전트 만들기

채용 공고 RAG 에이전트

- 아래 프롬프트를 평소 사용하는 AI(ChatGPT, Claude 등)에 입력하고, 생성된 코드를 VSCode에서 실행해보세요.

💡 프롬프트 예시

개발 환경은 Miniconda 가상환경(Python 3.11)이며 VSCode 터미널에서 실행합니다.
Python으로 LangGraph를 활용한 채용공고 RAG 에이전트를 만들어줘. 조건은 다음과 같아.

- .env 파일에서 OpenAI API Key를 불러오도록 구현
- 채용공고 내용을 job_posting.txt 파일로 저장하고 TextLoader로 불러오기
- RecursiveCharacterTextSplitter로 chunk_size=500, chunk_overlap=50으로 분할
- OpenAIEmbeddings로 벡터 변환 후 FAISS에 저장
- TypedDict로 RAGState 정의 (question, retrieved_docs, answer 필드 포함)
- retrieve_node: 벡터 데이터베이스에서 관련 문서 검색
- generate_node: 검색된 문서를 바탕으로 LLM이 답변 생성
- LangGraph로 retrieve → generate 순서로 그래프 구성
- gpt-4o-mini 모델 사용
- while True 루프로 사용자가 질문을 계속 입력할 수 있도록 구현
- "종료" 입력 시 프로그램 종료
- 필요한 라이브러리: langchain, langchain-openai, langgraph, faiss-cpu

더 발전시켜보기

채용 공고 RAG 에이전트

- **완성한 에이전트에 아래 기능 중 하나를 골라 직접 추가해보세요.**
앞에서 했던 것처럼 AI에게 추가 요청을 넣어서 코드를 발전시켜 나가면 됩니다.

추가 기능	프롬프트 힌트
여러 채용공고 동시 검색	"여러 개의 txt 파일을 불러와서 하나의 벡터 데이터베이스에 저장하도록 수정해줘"
검색 결과 개수 조절	"사용자가 검색할 chunk 개수를 입력할 수 있도록 수정해줘"
자소서 에이전트와 연결	"채용공고 RAG 에이전트로 검색한 결과를 바탕으로 자소서를 개선하는 기능을 추가해줘"

멀티 에이전트 협업 실습



멀티 에이전트란?

멀티 에이전트 협업 실습

- 지금까지 만든 에이전트는 모두 하나의 LLM이 모든 판단을 담당하는 싱글 에이전트 구조.
멀티 에이전트는 여러 개의 에이전트가 각자 역할을 나눠 협업하는 구조.
- 사람으로 비유하면, 싱글 에이전트는 혼자 모든 일을 처리하는 1인 기업이고, 멀티 에이전트는 각자 전문 분야가 있는 팀이 협업하는 회사와 같음. 팀원 각자가 자신의 전문 영역에서 최선의 결과를 내고, 팀장(Supervisor)이 전체 흐름을 조율해 최종 결과물을 만들어내는 방식.

	싱글 에이전트	멀티 에이전트
구조	LLM 하나가 모든 판단	여러 에이전트가 역할 분담
적합한 경우	단순하고 명확한 작업	복잡하고 다단계 작업
장점	구현 단순, 빠른 응답	각 에이전트가 전문성 발휘
단점	복잡한 작업에서 성능 저하	구현 복잡, 에이전트 간 조율 필요

- **멀티 에이전트가 필요한 경우:**
 - 하나의 LLM이 너무 많은 역할을 담당해 성능이 저하될 때
 - 각 단계별로 전문화된 판단이 필요할 때
 - 병렬로 여러 작업을 동시에 처리해야 할 때
 - 작업 규모가 커서 하나의 에이전트로 관리하기 어려울 때

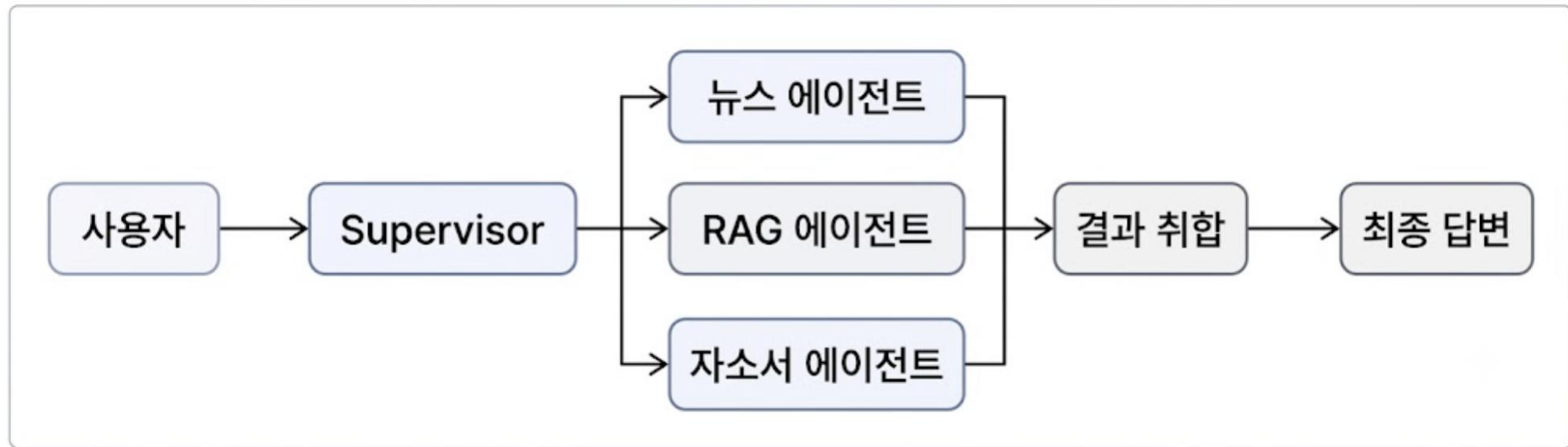
멀티 에이전트 패턴의 종류

멀티 에이전트 협업 실습

- 멀티 에이전트를 설계하는 방식은 크게 세 가지 패턴으로 나눌 수 있음. 작업의 성격과 복잡도에 따라 적합한 패턴을 선택하는 것이 중요.

- **Supervisor 패턴**

중앙의 Supervisor 에이전트가 사용자 요청을 분석해서 적절한 하위 에이전트에게 작업을 배분하고, 결과를 취합해 최종 응답을 생성. 가장 널리 사용되는 패턴으로 본 실습에서 구현하는 방식.

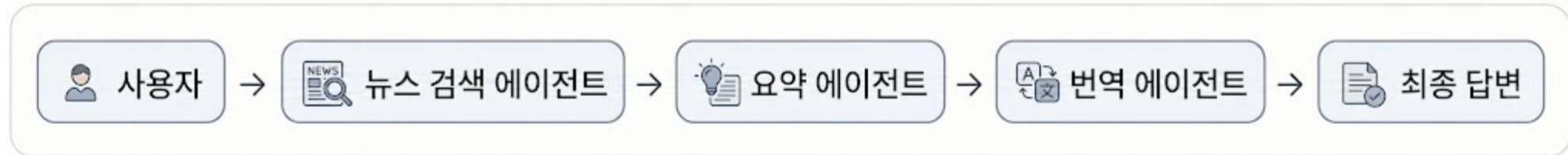


멀티 에이전트 패턴의 종류

멀티 에이전트 협업 실습

■ Pipeline 패턴

각 에이전트가 순서대로 작업을 처리하고 결과를 다음 에이전트에게 넘기는 방식. 작업 순서가 고정되어 있고 각 단계의 출력이 다음 단계의 입력이 되는 경우에 적합.

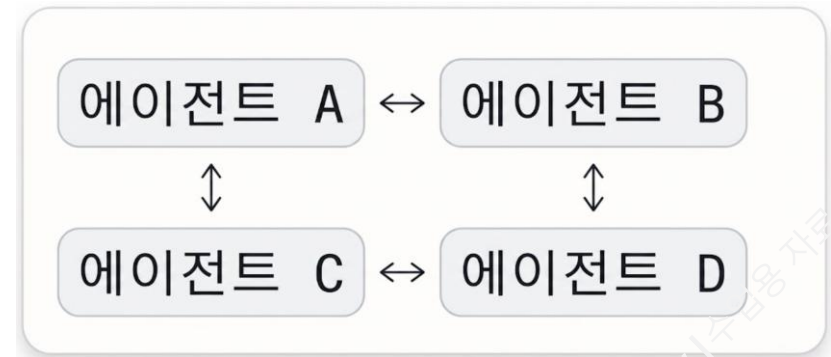


■ Peer-to-Peer 패턴

Supervisor 없이 에이전트들이 서로 직접 소통하며 협업하는 방식.

각 에이전트가 필요에 따라 다른 에이전트를 직접 호출.

유연하지만 구현이 복잡하고 흐름 추적이 어려움.



Supervisor 패턴이란?

멀티 에이전트 협업 실습

- 멀티 에이전트에서 가장 많이 사용되는 구조. Supervisor 에이전트가 사용자 요청을 분석해서 어떤 에이전트에게 작업을 맡길지 결정하고, 각 에이전트의 결과를 취합해 최종 응답을 생성.

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, List

class SupervisorState(TypedDict):
    user_input: str          # 사용자 입력
    next_agent: str          # 다음에 실행할 에이전트
    news_result: str         # 뉴스 검색 결과
    rag_result: str          # RAG 검색 결과
    resume_result: str       # 자소서 리뷰 결과
    final_answer: str        # 최종 취합된 답변

def supervisor_node(state: SupervisorState) -> SupervisorState:
    """ LLM이 사용자 요청을 분석해서 어떤 에이전트에게 작업을
    배분할지 결정 """
```

```
prompt = f"""
사용자 요청: {state["user_input"]}
아래 에이전트 중 가장 적합한 하나를 선택해서 이름만 답해줘.
- news: 뉴스 검색이나 채용 트렌드가 필요한 경우
- rag: 채용공고 분석이나 직무 관련 정보가 필요한 경우
- resume: 자소서 피드백이나 개선이 필요한 경우
- final: 위 세 가지에 해당하지 않는 경우
답변은 반드시 news, rag, resume, final 중 하나만 출력.
"""

response = llm.invoke(prompt)
state["next_agent"] = response.content.strip()
return state

def route_agent(state: SupervisorState) -> str:
    """Supervisor의 판단에 따라 다음 에이전트로 라우팅"""
    return state["next_agent"]
# "news" → 뉴스 검색 에이전트
# "rag" → RAG 에이전트
# "resume" → 자소서 리뷰 에이전트
# "end" → 최종 답변 생성
```


멀티 에이전트 그래프 구성

멀티 에이전트 협업 실습

- 앞에서 정의한 `supervisor_node`와 `route_agent`를 `LangGraph` 그래프에 연결해서 전체 멀티 에이전트 시스템을 완성하는 과정. 아래 코드는 참조용 코드로 구조를 이해하는 데 활용하세요.

```
...
def news_agent_node(state: SupervisorState) -> SupervisorState:
    """뉴스 검색 에이전트 노드 — 11~12차시 뉴스 검색 Tool 활용"""
    state["news_result"] = f"{state['user_input']} 관련 최신 뉴스 검색 결과"
    return state

def rag_agent_node(state: SupervisorState) -> SupervisorState:
    """RAG 에이전트 노드 — 21~22차시 RAG 파이프라인 활용"""
    state["rag_result"] = f"{state['user_input']} 관련 채용공고 분석 결과"
    return state

def resume_agent_node(state: SupervisorState) -> SupervisorState:
    """자소서 리뷰 에이전트 노드 — 17~18차시 자소서 Tool 활용"""
    state["resume_result"] = f"{state['user_input']} 자소서 피드백 결과"
    return state
...
```

멀티 에이전트 그래프 구성

멀티 에이전트 협업 실습

- 앞에서 정의한 `supervisor_node`와 `route_agent`를 `LangGraph` 그래프에 연결해서 전체 멀티 에이전트 시스템을 완성하는 과정. 아래 코드는 참조용 코드로 구조를 이해하는 데 활용하세요.

```
...
def final_node(state: SupervisorState) -> SupervisorState:
    """각 에이전트의 결과를 취합해 최종 답변 생성"""
    results = []
    if state["news_result"]:
        results.append(f"[뉴스 검색 결과]\n{state['news_result']}")
    if state["rag_result"]:
        results.append(f"[채용공고 분석]\n{state['rag_result']}")
    if state["resume_result"]:
        results.append(f"[자소서 피드백]\n{state['resume_result']}")
    state["final_answer"] = "\n\n".join(results)
    return state
# 그래프 구성
graph = StateGraph(SupervisorState)
graph.add_node("supervisor", supervisor_node)
graph.add_node("news", news_agent_node)
graph.add_node("rag", rag_agent_node)
graph.add_node("resume", resume_agent_node)
graph.add_node("final", final_node)
...
```

수업용 자료 | 무단 배포 금지

멀티 에이전트 그래프 구성

멀티 에이전트 협업 실습

- 앞에서 정의한 `supervisor_node`와 `route_agent`를 `LangGraph` 그래프에 연결해서 전체 멀티 에이전트 시스템을 완성하는 과정. 아래 코드는 참조용 코드로 구조를 이해하는 데 활용하세요.

```
...
graph.set_entry_point("supervisor")
# ↑ 항상 Supervisor에서 시작

graph.add_conditional_edges("supervisor", route_agent, {
    "news": "news",           # 뉴스 검색 에이전트로
    "rag": "rag",             # RAG 에이전트로
    "resume": "resume",       # 자소서 리뷰 에이전트로
    "final": "final"          # 바로 결과 취합으로
})

# 각 에이전트 완료 후 바로 결과 취합(final)으로 이동
# ↑ 다이어그램 구조와 동일: 에이전트 → 결과 취합 → 최종 답변
graph.add_edge("news", "final")
graph.add_edge("rag", "final")
graph.add_edge("resume", "final")
graph.add_edge("final", END)

app = graph.compile()
```

실습: 취업 코치 멀티 에이전트 만들기

멀티 에이전트 협업 실습

■ 실습의 목적:

- 지금까지 만들어온 에이전트들(뉴스 검색, RAG, 자소서 리뷰)이 하나의 시스템으로 통합되는 구조 체감
- Supervisor 패턴으로 여러 에이전트를 조율하는 방식 이해
- 멀티 에이전트 구조가 싱글 에이전트와 어떻게 다른지 비교
- AI가 생성한 코드에서 슬라이드 4, 5의 구조가 어떻게 구현되었는지 확인

🔍 확인해 볼 것

- Supervisor가 사용자 입력에 따라 어떤 에이전트를 선택하는지 확인
- 각 에이전트의 결과가 final_node에서 어떻게 취합되는지 확인
- LangSmith 대시보드에서 멀티 에이전트 흐름이 어떻게 시각화되는지 확인

실습: RAG 에이전트 만들기

멀티 에이전트 협업 실습

- 아래 프롬프트를 평소 사용하는 AI(ChatGPT, Claude 등)에 입력하고, 생성된 코드를 VSCode에서 실행해보세요.

💡 프롬프트 예시

개발 환경은 Miniconda 가상환경(Python 3.11)이며 VSCode 터미널에서 실행합니다.
Python으로 LangGraph를 활용한 취업 코치 멀티 에이전트를 만들어줘. 조건은 다음과 같아.

- .env 파일에서 OpenAI API Key를 불러오도록 구현
- TypedDict로 SupervisorState 정의 (user_input, next_agent, news_result, rag_result, resume_result, final_answer 필드 포함)
- 아래 노드 구현:
 - supervisor_node: 사용자 요청 분석 후 적절한 에이전트 선택
 - news_agent_node: 뉴스 검색 (임시 반환값 사용)
 - rag_agent_node: 채용공고 분석 (임시 반환값 사용)
 - resume_agent_node: 자소서 피드백 (임시 반환값 사용)
 - final_node: 각 에이전트 결과 취합 후 최종 답변 생성
- Supervisor → 조건 엣지로 각 에이전트 라우팅
- 각 에이전트 완료 후 바로 final_node로 이동 (Supervisor로 돌아가지 않음)
- gpt-4o-mini 모델 사용
- while True 루프로 사용자가 계속 질문할 수 있도록 구현
- "종료" 입력 시 프로그램 종료

더 발전시켜보기

멀티 에이전트 협업 실습

- **완성한 에이전트에 아래 기능 중 하나를 골라 직접 추가해보세요.**
앞에서 했던 것처럼 AI에게 추가 요청을 넣어서 코드를 발전시켜 나가면 됩니다.

추가 기능	프롬프트 힌트
에이전트 선택 결과 출력	"Supervisor가 어떤 에이전트를 선택했는지 사용자에게 안내 메시지 출력해줘"
결과를 파일로 저장	"최종 취업 코칭 결과를 txt 파일로 저장하는 기능 추가해줘"
실제 Tool 연동	"임시 반환값 대신 뉴스 검색 Tool과 RAG 파이프라인을 실제로 연동해줘"

오늘 하루도 수고하셨습니다